Creating a Bell State

A Bell state is a quantum state of two qubits that are maximally entangled, demonstrating the quintessential quantum properties of superposition and entanglement.

Step 1: Import Necessary Modules

```
from qiskit import QuantumCircuit, Aer, transpile, assemble
from qiskit.visualization import plot_histogram
from qiskit.providers.ibmq import least_busy
```

Step 2: Initialise the Circuit

Create a quantum circuit with 2 qubits and 2 classical bits for measurement.

```
qc = QuantumCircuit(2, 2)
```

Step 3: Apply Quantum Gates

Apply a Hadamard gate to the first qubit to create superposition.

```
qc.h(0)
```

Apply a CNOT gate with the first qubit as the control and the second as the target to entangle them

```
qc.cx(0, 1)
```

Step 4: Measure the Qubits

Measure the qubits and store the results in the classical bits.

```
qc.measure([0,1], [0,1])
```

Step 5: Run the Circuit on a Simulator

```
Use Aer's qasm_simulator to simulate the circuit.
simulator = Aer.get_backend('qasm_simulator')
compiled_circuit = transpile(qc, simulator)
job = simulator.run(assemble(compiled_circuit))
result = job.result()
```

Step 6: Visualise the Results

Plot the measurement outcomes

```
counts = result.get_counts(qc)
plot_histogram(counts)
```

The histogram displays the results of the quantum circuit execution, showcasing the entangled state where the outcomes are correlated.

This simple quantum program demonstrates the core principles of quantum mechanics in action. By creating a Bell state, developers can visualise the entanglement and superposition of qubits, foundational concepts for more complex quantum algorithms.

Quantum Software Development Kits (SDKs) and Libraries

After establishing the foundational knowledge of quantum programming languages and setting up a quantum programming environment, we delve deeper into the tools that further empower quantum developers: Quantum SDKs and Libraries.

The quantum computing ecosystem is rich with specialised SDKs and libraries designed to simplify the development of quantum algorithms and applications. These tools offer abstractions and functionalities that go beyond basic quantum programming languages, enabling more efficient and effective quantum software development.

Overview of Quantum SDKs

- **Strawberry Fields:** Developed by Xanadu, Strawberry Fields is a Python library for simulating and optimising quantum optics. It provides tools for designing, simulating, and optimising quantum optical circuits, making it a cornerstone for photonic quantum computing.
- **Ocean by D-Wave:** Targeting quantum annealing technology, Ocean provides a comprehensive suite for developing applications on D-Wave's quantum annealers. It supports various high-level abstractions for formulating and solving optimization problems on quantum hardware.

Featured Quantum Libraries

- **PennyLane:** Also from Xanadu, PennyLane is a cross-platform Python library for quantum machine learning, automatic differentiation, and optimization of hybrid quantum-classical computations. PennyLane uniquely facilitates the integration of classical machine learning libraries with quantum hardware and simulators.
- **QTensor:** QTensor is a quantum circuit simulator with a focus on tensor network simulations. It's designed for large-scale quantum circuit simulations, providing insights into how quantum algorithms perform at scale.

Integrating SDKs and Libraries

Most quantum SDKs and libraries can be installed via pip. For example, to install PennyLane, one would use:

```
pip install pennylane
```

After installation, developers can quickly start experimenting with quantum circuits. For instance, using PennyLane to create a simple quantum circuit might involve:

```
import pennylane as qml
dev = qml.device('default.qubit', wires=1)

@qml.qnode(dev)
def circuit(x):
    qml.RX(x, wires=0)
    return qml.expval(qml.PauliZ(0))
```